

Prototype Lab Report

ME 2900

Instructor: Kai Vogeler

Date of Submission: 10/14/2025

Shaun Sekura

I. Introduction

This prototype is intended to be used as an infrared controlled desk fan with a display. The intentions and motivations behind the prototype were to be used as a desk fan alternative to my previous one. I did not have control over several aspects of my previous desk fan such as precise fan speed and visually seeing what was happening with my fan. Building this prototype gave me a nice desk fan with the functionality and customization that I desired. I wanted to accomplish this custom fan with a display to improve my studying habits and other aspects of my day to day life. After more iterations of this prototype, the final product might be more compact and have a feature to rotate around at a set speed to disperse the blown air.

II. Discussion

Idea Development Process

I started planning my solution by determining what I most desired in a custom fan. I figured these to be controllable fan speed and a visual display. I then implemented these parts as a DC motor and an IR receiver and remote and built the other parts around these core components. Other solutions I was brainstorming were a servo motor to rotate the fan and a potentiometer to manually change the fan speed, but I decided against either of these due to the tight deadline and clunkiness this would bring to the project.

Other Solutions Considered

I considered using a keypad instead of an IR receiver and remote, but I decided against it to allow the remote to be used wirelessly instead of having to be next to the fan to operate it. For the programming logic, I considered using for loops to change the fan speed instead of using analogWrite to make the fan change speed smoother. I decided against this because it was a lot of extra code for minimal performance change. For the physical elements, I was considering making the fan slanted at an angle instead of horizontal, but I decided against it because I personally prefer the fan to be at a horizontal instead of at an angle.

Discussion of Prototype Operation/Features

When the code is started, the fan will display a welcome message. The fan will then start up, ramping up the fan speed to Level 1. From this point on, the user has complete control over operating the fan. The buttons 1, 2, and 3 on the IR remote correspond to Level 1, Level 2, and Level 3 fan speed, with fan speed increasing linearly across the levels. When one of these three buttons are pressed, the display will show a message to wait a few seconds, the LED will briefly flash, the piezo buzzer will make a sound, and then the fan will proceed with changing the fan speed to the selected level. If a button other than these three are pressed, the display will show a message stating an unknown command was registered and the fan will not change speed. The pushbutton functions as an emergency stop. When pressed, the fan will completely stop, the LED will be turned on, and the piezo buzzer will make a noise. When pressed again, the LED will turn off, the piezo buzzer will make a noise, and the fan will resume at the last selected level.

The code starts by initializing the libraries and variables necessary for the code to function. The display and motor are then started, setting the motor and display to Level 1. Several functions are then initialized, including the `changeInSpeed`, `beep`, and `changeFan` commands, which state the change in speed of the fan when a button is pressed, set the piezo buzzer to beep at a 2 kHz tone, and combines redundant parts of the code that changes the speed of the fan, respectively. The loop then starts, initializing the portion of the code that changes the fan speed based on the signals from the IR remote. The code uses the CMD commands of the remote to determine when a button is pressed, using '0C' for 1, '18' for 2, and '5E' for 3. As mentioned before, the code uses if and else-if logic to figure out if the button pressed was 1, 2, 3, or some other button and controls the system accordingly. The last section of the code is the pushbutton that functions as an emergency stop. If the button is read as LOW, or the button is pressed, the code will check if the `motorState` is 0 or 1. The `motorState` starts at 1, so the first time the button is pressed, the code will change the `motorState` to 0, turn on the LED, the piezo buzzer will beep for 120 ms, and the display will say the fan is off. When pressed again, the `motorState` will be set to 1 and everything will revert back to how it was before the button was pressed.

III. Conclusion

Difficulties

One difficulty I had in creating this prototype was reaching the required amounts of points for implementing components into the system. When I implemented the core components, the IR receiver/remote, LCD display, and DC motor, I had seven points and was unsure where I would get the final 3 points. To resolve this issue, I first tried to implement a relay into the system which did not work. I then implemented several 1 point components, being a pushbutton, LED, and piezo buzzer, to function as an emergency stop system which worked very well when implemented into the system.

Interesting/Successful Feature

One successful feature of my system that I really liked was the IR receiver/remote. I was very proud of implementing this component because I spent a long time researching it and figuring out exactly how to make it interact with the code in the way I wanted. Finally learning how to use CMD commands to control output of different components really put my problem-solving skills to the test, and I enjoyed the process.

Changes/Improvements

One aspect of my prototype I would have improved given more time and resources is the casing for the fan and the system. If I had more time and resources, I could have made a more compact and sturdy casing, possibly using sheet metal or 3D printing. This could be achieved using a design with the arduino as the base, the wiring running up the center, and the fan at top, making a more compact and efficient design.

Appendix

Pictures

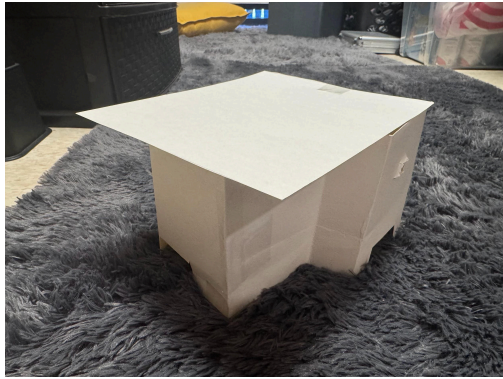


Figure 1: The prototype inside the casing

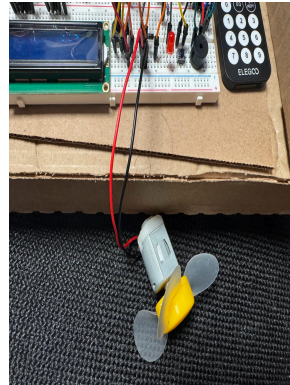


Figure 2: DC Motor

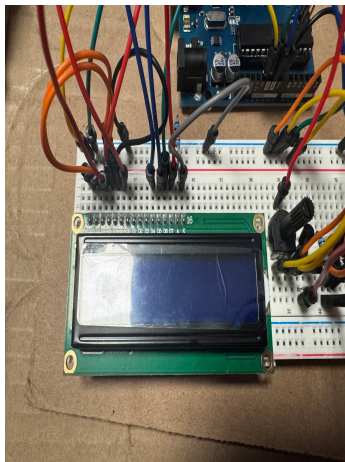


Figure 3: LCD Display



Figure 4: IR Receiver and Remote

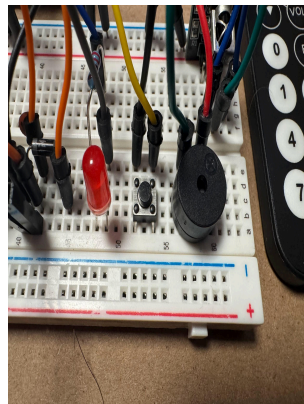
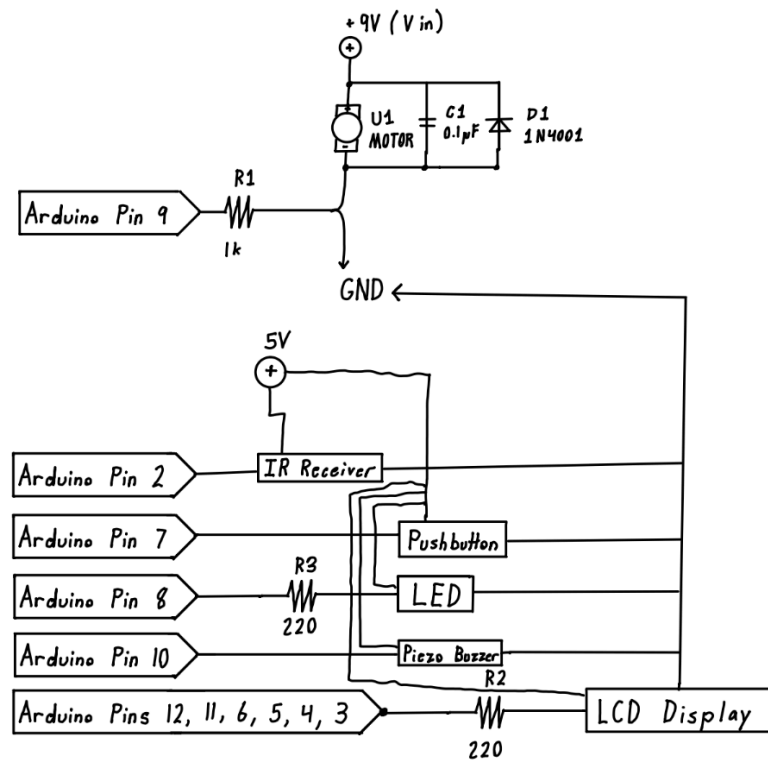


Figure 5: Emergency Stop (Pushbutton, LED, Piezo Buzzer)

Circuit Schematics



Arduino Sketches

```
#define IR_SMALLD_NEC
#include <IRsmallDecoder.h>
#include <LiquidCrystal.h>

IRsmallDecoder irDecoder(2); //IR Receiver
irSmallD_t irData;
LiquidCrystal lcd(12, 11, 5, 4, 3, 6);

const int MOTOR = 9;    // Motor
const int BUTTON = 7;    // Pushbutton
const int LED = 8;      // LED
const int BUZZER = 10;   // Piezo buzzer

int i = 0;
int j = 0;
int motorState = 1;      // 1 = ON, 0 = OFF

int changeInSpeed(int x, int y){
    int result = y-x;
    return result;
}

void setup() {
    Serial.begin(9600);
    Serial.println("Waiting for a NEC remote control IR signal...");
    Serial.println("held\t addr\t cmd");
```

```

pinMode(MOTOR, OUTPUT);
pinMode(LED, OUTPUT);
pinMode(BUZZER, OUTPUT);
pinMode(BUTTON, INPUT_PULLUP);

// Initialize Display
lcd.begin(16, 2);
lcd.print("Welcome to fan");
delay(2000);
lcd.begin(16, 2);
lcd.print("Starting...");

// Start motor
for (int i = 0; i < 76; i++) {
    analogWrite(MOTOR, i);
    delay(50);
}
i = 75;
analogWrite(MOTOR, i);
lcd.begin(16, 2);
lcd.print("Level 1");
}

// Beep function
void beep(int duration) {
    tone(BUZZER, 2000); // 2kHz tone
    delay(duration);
    noTone(BUZZER);
}

// Changing fan speed function
void changeFan() {
    digitalWrite(LED, HIGH); // LED ON while changing
    beep(80);
    lcd.begin(16, 2);
    lcd.print("Wait...");
    delay(2000);
    analogWrite(MOTOR, i);
    digitalWrite(LED, LOW); // LED OFF when done
    lcd.begin(16, 2);
}

void loop() {
    // Select button
    if (irDecoder.dataAvailable(irData)) {
        Serial.print(irData.keyHeld);
        Serial.print("\t ");
        Serial.print(irData.addr, HEX);
        Serial.print("\t ");
        Serial.println(irData.cmd, HEX);

        if (irData.cmd == 0x0C) { // Level 1
            int num1 = i;
            i = 75;
            int num2 = i;
            changeFan();
            int SpeedChange = changeInSpeed(num1, num2);

```



```

        lcd.print("Level 1");
        Serial.print("The change in speed is: ");
        Serial.println(SpeedChange);
    }

    else if (irData.cmd == 0x18) { // Level 2
        int num1 = i;
        i = 150;
        int num2 = i;
        changeFan();
        int SpeedChange = changeInSpeed(num1, num2);
        lcd.print("Level 2");
        Serial.print("The change in speed is: ");
        Serial.println(SpeedChange);
    }

    else if (irData.cmd == 0x5E) { // Level 3
        int num1 = i;
        i = 225;
        int num2 = i;
        changeFan();
        int SpeedChange = changeInSpeed(num1, num2);
        lcd.print("Level 3");
        Serial.print("The change in speed is: ");
        Serial.println(SpeedChange);
    }

    else {
        // Unknown command
        lcd.begin(16, 2);
        lcd.print("Unknown cmd:");
        lcd.setCursor(0, 1);
        lcd.print(irData.cmd, HEX);
        beep(150);
    }
}

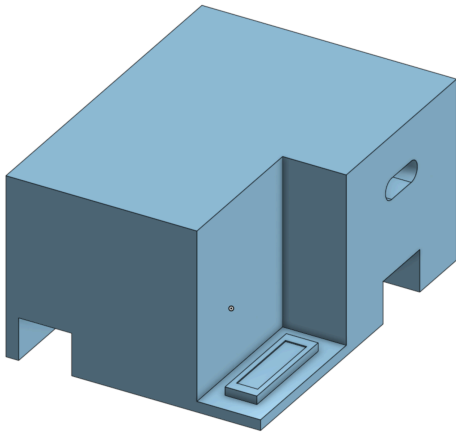
// Pushbutton e stop
if (digitalRead(BUTTON) == LOW) { // button pressed
    if (motorState == 0) {
        // Turn fan on
        motorState = 1;
        analogWrite(MOTOR, i);
        digitalWrite(LED, LOW);
        beep(60);
        lcd.begin(16, 2);
        lcd.print("Fan ON");
    }
    else {
        // Turn fan off
        motorState = 0;
        analogWrite(MOTOR, 0);
        digitalWrite(LED, HIGH);
        beep(120);
        lcd.begin(16, 2);
        lcd.print("Fan OFF");
    }
    delay(300); // debouncing
}
}

```

Libraries:

IRSmallDecoder: <https://github.com/LuisMiCa/IRsmallDecoder>

LiquidCrystal: <https://docs.arduino.cc/libraries/liquidcrystal/>

SOLIDWORKS Rendering:**Video:**

https://youtu.be/Z13_E-RVOWQ